

```
        raise WizCoinException('cannot multiply with negative integers')

    # The WizCoin class creates mutable objects, so do NOT create a
    # new object like this commented-out code:
    #return WizCoin(self.galleons * other, self.sickles * other, self.knuts *
other)

    # We modify the `self` object in-place:
    self.galleons *= other
    self.sickles *= other
    self.knuts *= other
    return self # In-place dunder methods almost always return self.
```

---

The `WizCoin` objects can use the `+=` operator with other `WizCoin` objects and the `*=` operator with positive integers. Notice that after ensuring that the other parameter is valid, the in-place methods modify the `self` object in-place rather than creating a new `WizCoin` object. Enter the following into the interactive shell to see how the augmented assignment operators modify the `WizCoin` objects in-place:

---

```
>>> import wizcoin
>>> purse = wizcoin.WizCoin(2, 5, 10)
>>> tipJar = wizcoin.WizCoin(0, 0, 37)
❶ >>> purse + tipJar
❷ WizCoin(2, 5, 46)
>>> purse
WizCoin(2, 5, 10)
❸ >>> purse += tipJar
>>> purse
WizCoin(2, 5, 47)
❹ >>> purse *= 10
>>> purse
WizCoin(20, 50, 470)
```

---

The `+` operator ❶ calls the `__add__()` or `__radd__()` dunder methods to create and return new objects ❷. The original objects operated on by the `+` operator remain unmodified. The in-place dunder methods ❸ ❹ should modify the object in-place as long as the object is mutable (that is, it's an object whose value can change). The exception is for immutable objects: because an immutable object can't be modified, it's impossible to modify it